

Tareas cron escribibles

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

cron ejecuta tareas programadas, a menudo como root. Si una de esas tareas llama a un script, binario o ruta que tu usuario puede escribir (o si usa comodines o rutas relativas manipulables), puedes inyectar tu propio comando y esperar a que cron lo ejecute con privilegios. Es el privesc-modelo aplicado al tiempo.

Los admins programan mantenimiento (backups, limpiezas) como root y a veces dejan el script en una ruta escribible por otros, o usan comodines (tar *) que un atacante puede secuestrar creando ficheros con nombres especiales. cron lo ejecuta tal cual, sin validar quien toco el script.

Modelo mental

```
Entrada controlada -> transformaciones -> validación -> componente final
                    -> sink -> efecto observable -> decisión
```

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Cuando sudo (sudo-abuse) y SUID (suid) no dieron nada. cron es mas lento porque hay que esperar a que la tarea corra, pero es un vector muy comun en rooms.

Cómo identificarla

- Un script referenciado en cron es escribible por tu usuario o su grupo.
- La tarea usa una ruta relativa o un comodin (*) en un directorio donde puedes crear ficheros.
- Aparecen procesos que arrancan cada pocos minutos (pspy lo revela) corriendo como root.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Comprueba con ls -l si el script que ejecuta root es escribible por tu usuario o grupo.

Evidencia esperada: Permiso de escritura (w) para ti en el script; tras inyectar tu payload y esperar el ciclo de cron, una shell/binario root.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.

3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
cat /etc/crontab; ls -la /etc/cron.*
```

Objetivo y contexto: Muestra las tareas programadas del sistema y con que frecuencia corren. Es el primer sitio donde mirar que ejecuta root de forma automática.

Resultado esperado: Líneas con horario, usuario (root) y el comando/script. Anota los scripts que corren como root: el siguiente paso es ver si puedes escribirlos.

Capa 2: prueba documentada

```
ls -l /ruta/al/script_de_cron.sh
```

Objetivo y contexto: Comprueba si el script que ejecuta root es escribible por ti. Ese permiso de escritura es justo la frontera de confianza rota que necesitas.

Resultado esperado: Los permisos y el dueño. Si tu usuario o un grupo tuyo tiene w, puedes inyectar comandos. Si no, busca comodines o rutas relativas manipulables.

Anatomía de los payloads

La primera prueba es `cat /etc/crontab; ls -la /etc/cron.*`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Permiso de escritura (w) para ti en el script; tras inyectar tu payload y esperar el ciclo de cron, una shell/binario root.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Lee las tareas programadas del sistema y de otros usuarios.
- Comprueba permisos de los scripts/binarios que invocan (`ls -l`): busca los escribibles.
- Inyecta tu payload (una reverse shell o el bit SUID a bash) en el punto controlable.
- Espera a que cron ejecute y recoge tu shell/binario root.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- Ninguna tarea root referencia un script o ruta que puedas escribir.

- Los scripts invocados usan rutas absolutas y permisos correctos (root:root, 700) sin comodines.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Si root ejecuta periodicamente un script o binario que tu puedes modificar, tu código correrá como root. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y condiciones.
2. Identificar entrada, componente y permisos.

3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook \(cron-abuse\)](#)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.